Decommissioning Services: The Art of Turning Off

Published on June 2, 2024

Webstack Builders

Table of Contents

Spinning up a new service is hard — getting infrastructure, CI/CD, and monitoring in place takes real effort. But turning off an old service? That's harder. The asymmetry makes sense: creating something new has bounded risk (it either works or it doesn't), but deleting something has unbounded risk (you won't know what breaks until it breaks). Nobody gets promoted for removing things. And the one time someone deleted a "dead" service that turned out to power a VP's quarterly dashboard, the story echoed through the organization for years.

So services accumulate. They sit there, burning money, expanding your attack surface, and confusing new engineers who wonder why the architecture diagram has 47 boxes when only 12 seem to do anything. This article covers how to find out what's actually used, how to turn things off safely, and how to communicate so nobody gets surprised at 3 AM.

The Decommissioning Problem

Every organization I've worked with has more zombie services than they realize. These aren't services that are completely dead — those are easy. Zombies are the ones that *might* be dead. They receive occasional traffic that could be health checks or could be production workloads. They haven't been deployed in months, but maybe that's because they're "stable." The owning team dissolved in a reorg, but surely someone took over. The uncertainty is what keeps them running.

The Cost of Not Decommissioning

Zombie services aren't free. The direct costs are visible in your cloud bill: compute, storage, and licenses for dependencies nobody remembers adding. But the indirect costs are often larger and harder to measure.

Cost Category	Monthly Range	What Gets Missed
Compute	$100 - $5,000	Idle instances still bill full price
Storage	$50 - $2,000	Data grows forever without cleanup
Licenses	$0 - $10,000+	Per-seat and per-instance fees
Security patching	$200 - $2,000	Unpatched services are vulnerable services

Cost Category	Monthly Range	What Gets Missed
Cognitive overhead	$500 - $5,000	Every zombie confuses new engineers

Direct and indirect costs of zombie services.

Beyond the line items in your cloud bill, security risk compounds quietly. A service that hasn't been deployed in six months is a service that hasn't received security patches in six months. It's running old dependencies with known CVEs, and nobody's watching the vulnerability scanner for it because nobody owns it.

Then there's the compliance surface area. Every service that handles data is a service your auditors want to examine. Zombies expand your SOC 2 scope without providing any value.

> **⚠ WARNING**
>
> Every zombie service makes the next decommissioning harder. They create phantom dependencies, confuse new engineers about what's actually production-critical, and accumulate technical debt that blocks migrations.

Why Decommissioning Is Scary

The fear cascade is predictable. "What if someone uses it?" leads to "What if it breaks something?" which leads to "Who owns the decision?" which leads to "What if we need it later?" which ends at "It's not hurting anything, let's just leave it."

Each step is rational in isolation. You *don't* have visibility into usage. Dependencies *are* unknown. Authority *is* unclear. You *might* need it. And the costs *are* invisible because nobody's tracking them per-service.

The default is always "leave it running" because turning something off requires courage and knowledge that leaving it alone doesn't require. This article is about getting that knowledge so the courage part is easier.

Signs a Service Should Be Decommissioned

Not every quiet service is a decommissioning candidate. Some services are legitimately low-traffic — the annual compliance report generator doesn't need to handle requests in July. But certain signals, especially in combination, indicate a service worth investigating:

Zero traffic	for 30+ days (not just low traffic-nearly zero)
No deployments	in 6+ months
No code changes	in 12+ months
Deprecated dependencies	that are end-of-life
No valid owner	because the team dissolved or left the company
Replaced by	a newer service that does the same thing
One-time migration	that finished months ago but the job is still running

Any one of these might be fine. Three or more together? That's a candidate. I score services by combining weighted signals — zero traffic scores highest, no deployments lower — and anything above a threshold goes on the review list.

The Scream Test: Controlled Failure

The scream test is exactly what it sounds like: turn something off and wait for someone to scream. It's the most reliable way to discover if anyone actually uses a service, because the consumers who don't know they're consumers will reveal themselves when things break.

But you can't just flip the switch and hope. A well-designed scream test gives consumers time to notice problems at each stage, provides clear information about what's happening and who to contact, and has automatic rollback triggers for when the screaming gets too loud.

Scream Test Design

A proper scream test has four phases, each lasting about a week. The goal is progressive degradation that surfaces dependencies without causing lasting damage.

1. **Phase 1: Soft degradation.**
Add latency to every request-start at 100ms, increase to 500ms, then 1 second. Services that depend on you in their critical path will start seeing timeouts and SLA violations. Their monitoring should catch it. If nobody notices 1 second of added latency, that's a strong signal the dependency isn't critical.

2. **Phase 2: Intermittent failures.**
Start failing a percentage of requests with 503 Service Unavailable and a Retry-After header. Begin at 1%, move to 5%, then 10%. Well-behaved clients will retry and succeed. Poorly-behaved clients will surface as consumer error rates spike. Either way, you learn who's calling you.

3. **Phase 3: Hard shutdown.**
Return 410 Gone for all requests. This is HTTP's way of saying "this resource existed but doesn't anymore, and it's not coming back." Log every caller at this stage-anyone still hitting you after weeks of deprecation warnings either isn't paying attention or has an integration nobody knew about.

4. **Phase 4: Complete removal.**
After a quiet period (I use two weeks), remove the service from the load balancer and delete the compute resources. Keep data archives for now-you're not done until you're sure nobody's screaming.

> **INFO**
>
> Send deprecation notices to every communication channel 2-4 weeks before starting Phase 1. Email, Slack, team standups — anywhere consumers might see it. Some will self-identify, saving you discovery effort during the test.

Throughout all phases, add deprecation headers to every response. The `Deprecation` and `Sunset` headers are standardized (RFC 8594), and good HTTP clients will log warnings when they see them. Include a link to your migration guide and a contact email for questions.

Implementing Soft Shutdown Responses

The key to a good scream test is returning helpful responses, not just errors. When you fail a request, tell the caller *why* it failed and *what to do about it*.

scream-test-middleware.ts

```
// Express middleware for scream test phases
function screamTestMiddleware(config: ScreamTestConfig) {
  return async (req: Request, res: Response, next: NextFunction) => {
    const phase = getCurrentPhase(config)

    // Always add deprecation headers, even when requests succeed
    res.set({
      'Deprecation': 'true',
      'Sunset': config.sunsetDate.toISOString(),
      'Link': `<${config.migrationGuide}>; rel="deprecation"`,
      'X-Decommission-Phase': phase.name,
      'X-Decommission-Contact': config.contactEmail
    })

    // Log every request for consumer discovery
    logScreamTestRequest({
      service: config.serviceId,
      phase: phase.name,
      caller: identifyCaller(req), // see below
      endpoint: req.path,
      userAgent: req.headers['user-agent'],
      timestamp: new Date()
    })

    switch (phase.behavior.type) {
      case 'add-latency':
        await sleep(phase.behavior.latencyMs)
        return next()

      case 'fail-percentage':
```

```
          if (Math.random() < phase.behavior.percentage / 100) {
            return res.status(503).json({
              error: 'Service is being decommissioned',
              migrationGuide: config.migrationGuide,
              retryAfter: 60,
              phase: phase.name
            })
          }
          return next()

        case 'return-gone':
          return res.status(410).json({
            error: 'Service has been decommissioned',
            migrationGuide: config.migrationGuide,
            contactEmail: config.contactEmail,
            alternatives: config.alternatives
          })
      }
    }
  }
```

Express middleware that implements scream test phases with helpful error responses.

The `identifyCaller` function is critical for discovery. Extract everything you can from the request: IP address, service name headers (if your mesh requires them), user agent, trace IDs, API keys, mTLS client certificates. The more ways you can identify callers, the easier the "who's still using this?" conversation becomes.

Scream Test Monitoring

You need a dashboard that answers three questions in real-time: Who's calling? How often? And is anyone complaining?

Metric	What It Tells You
Request rate by caller	Which services are still dependent
Unique caller count	Total number of integrations to migrate

Metric	What It Tells You
Consumer error rate	Whether your test is causing downstream problems
Tickets/escalations	Human impact beyond automated monitoring
Days until sunset	Countdown for urgency in communications

Key metrics to track during a scream test.

Set up automatic rollback triggers. If consumer error rates spike above a threshold, or if a critical alert fires, pause the test and investigate. The goal is discovery, not damage. A scream test that causes an outage defeats the purpose — you want to find unknown dependencies **before** they become incidents.

Traffic Analysis and Consumer Discovery

The scream test is the final answer to "who uses this?" But you don't want to go in blind. Before you start degrading a service, spend a few weeks doing passive discovery — analyzing traffic patterns, tracing requests, and checking data dependencies. The more consumers you can identify upfront, the fewer surprises during the test.

Passive Traffic Analysis

Your access logs and service mesh telemetry already contain the information you need. The challenge is extracting it in a useful form.

Start by grouping requests by caller. The identification method depends on your infrastructure: IP addresses work for internal services with stable IPs, service name headers work if your mesh requires them, mTLS client certificates are ideal when available, and API keys identify external consumers.

For each caller, classify the traffic pattern. This tells you what kind of dependency you're dealing with:

#	Pattern	Characteristics	Implication
1	Scheduled	Very regular intervals (±10% variance)	Batch job, can probably tolerate downtime windows

#	Pattern	Characteristics	Implication
2	Batch processing	Bursts followed by silence	ETL pipeline, may only run monthly
3	Real-time dependency	Consistent throughout the day	Critical path, needs careful migration
4	Irregular	No discernible pattern	Often human-triggered or debug traffic

Traffic patterns and what they tell you about consumer dependencies.

Skip callers with fewer than one request per day — those are usually health checks, monitoring probes, or one-off debugging sessions. Focus your energy on the high-volume consumers and the ones with real-time patterns.

Distributed Tracing for Dependency Discovery

If you have distributed tracing (Jaeger, Zipkin, or AWS X-Ray), you can discover dependencies you'd never find through traffic analysis alone. Traces show you *who calls you* (upstream dependencies) and *who you call* (downstream dependencies).

Query any traces that include your service over a 30-day window. For each trace, find the spans belonging to your service and walk the tree in both directions. The parent span tells you who initiated the call. The child spans tell you what services you depend on.

This matters for decommissioning because your service might be a transitive dependency. ServiceA calls ServiceB calls your service — and ServiceA's team has no idea they depend on you because they've never called you directly. The trace data reveals these hidden chains.

> **INFO**
>
> Trace sampling can miss low-volume callers. If you sample at 1%, a consumer that calls you 50 times per day might not appear in your trace data at all. Supplement tracing with access logs for complete coverage.

Database Query Analysis

A service might have zero HTTP traffic but still own critical data. Before decommissioning, check what database tables or collections the service touches.

If you're using PostgreSQL with query tagging (comments in SQL that identify the calling service), you can query `pg_stat_statements` to find every table the service reads from or writes to. The distinction matters: a service that only reads from a table can be removed without affecting the data, but a service that writes to a table might be the only thing populating it.

pg-table-dependencies.sql

```sql
-- Find tables accessed by a service (requires query tagging)
SELECT
  regexp_matches(query, 'FROM\s+(\w+)', 'gi') AS tables_read,
  regexp_matches(query, 'INSERT INTO\s+(\w+)', 'gi') AS tables_written,
  calls,
  total_exec_time
FROM pg_stat_statements
WHERE query LIKE '%/* service:legacy-user-lookup */%'
ORDER BY calls DESC;
```

PostgreSQL query to identify table dependencies from `pg_stat_statements`.

> **⚠ WARNING**
>
> A service might have zero traffic but still own critical data. Always check what tables the service writes to before decommissioning — you might be about to orphan a dataset that other services depend on.

Communication and Stakeholder Management

The technical work of decommissioning is straightforward compared to the human work. You can automate traffic analysis and rollback triggers, but you can't automate convincing a product manager that their "critical integration" can wait two weeks for migration support.

Good communication prevents most escalations. Bad communication — or no communication — guarantees them.

Deprecation Announcement Template

The announcement is the foundation of your communication plan. It needs to answer every question a consumer might have, because the questions you don't answer become Slack messages and escalation tickets.

A complete announcement includes: what's being decommissioned and when, why it's being decommissioned (cost savings, security, replacement available), the timeline with specific dates for each phase, the migration path with links to documentation, who to contact for help, and what happens if someone isn't ready by the deadline.

deprecation-notice-template.md

```
# Service Deprecation Notice: [Service Name]

**Summary:** [Service Name] will be decommissioned on [Date].

## Timeline
- Today: Deprecation announced
- [Date + 2 weeks]: Soft degradation begins (added latency)
- [Date + 4 weeks]: Intermittent failures (10% of requests)
- [Date + 6 weeks]: Service returns 410 Gone
- [Date + 8 weeks]: Infrastructure removed

## If You Use This Service
1. Check if you're affected: [link to consumer list or self-check tool]
2. Migrate to: [alternative service] using [migration guide link]
3. Contact us: [Slack channel] or [email] for migration help

## FAQ
**Not ready by the deadline?** Contact us 2+ weeks before to discuss an extension.
**What happens to the data?** Archived to [location], retained for [period].

## Support
- Questions: #decommissioning-support
- Office hours: [calendar link] (Tuesdays 2-3pm)
- Escalation: [manager name] for business-critical blockers
```

Deprecation announcement template covering the essential information consumers need.

Post this announcement in multiple channels: email to known consumers, Slack in relevant team channels, mention in all-hands or engineering syncs, and a wiki page that becomes the canonical reference. Repetition isn't annoying here — it's necessary. People miss announcements.

Escalation Handling

Despite your best communication efforts, consumers will emerge during the scream test who didn't see the announcements or ignored them. You need a process for handling these escalations that doesn't derail the entire decommissioning.

Escalations fall into four categories, each with a standard response:

Escalation Type	Response	Timeline Impact
Information request	Send migration guide, offer office hours	None
Migration assistance	Pair with consumer on migration	None if quick, extension if complex
Extension request	Evaluate impact, grant 1-2 weeks if justified	Minor delay
Blocking escalation	Pause scream test, investigate immediately	Potential major delay

Escalation categories and standard responses.

For blocking escalations—"this will cause an outage for 10,000 users if you proceed"—pause the scream test immediately. You don't want to be the team that caused an incident because you were too committed to your own timeline. Investigate, understand the true impact, and adjust the plan.

Grant extensions based on demonstrated impact, not urgency theater. Compliance concerns get automatic extensions (you don't want to explain to auditors why you broke a regulatory integration). Revenue impact above a threshold gets an extension. "We're busy" doesn't get an extension — everyone's busy.

> **INFO**
>
> Keep an audit trail of every escalation and resolution. When leadership asks why the decommissioning took 12 weeks instead of 8, you want receipts showing which teams requested extensions and why.

Communication Timeline

Spread your communications across the decommissioning timeline. Front-loading all communication in week one means everyone forgets by week four.

- **Week -4 (announcement):**
  Initial deprecation notice across all channels. Create the wiki page. Email known consumers directly.
- **Week -2 (reminder + scream test start):**
  Reminder that Phase 1 begins. Direct outreach to consumers who haven't acknowledged. Start soft degradation.
- **Week -1 (status update):**
  Weekly update on migration progress. List remaining consumers publicly (social pressure works). Remind about office hours.
- **Week 0 (shutdown):**
  24-hour warning. Final call for escalations. Execute shutdown during low-traffic window.
- **Week +1 (post-shutdown):**
  Confirm successful shutdown. Publish lessons learned. Report cost savings to justify the effort.

Shutdown Execution

With stakeholders aligned and consumers migrated (or at least warned), you're ready for the actual shutdown. This is the part where things can still go wrong if you rush it. This is the part where things can still go wrong if you rush it.

Shutdown Checklist

A checklist sounds bureaucratic, but it prevents the “oh no, we forgot to revoke the service account credentials” moment three weeks later. Break it into four phases:

Pre-shutdown (before you touch anything):

1. All known consumers migrated and confirmed
2. Scream test completed without blocking escalations
3. Data backup completed and verified
4. Rollback plan documented and tested
5. Stakeholder sign-off obtained
6. Monitoring alerts silenced or removed
7. On-call team notified

Shutdown (the actual turning off):

8. Remove from load balancer (stops new traffic)
9. Scale deployment to zero replicas
10. Update DNS if applicable (point to deprecation page or remove)
11. Stop scheduled jobs and cron tasks
12. Revoke service account credentials

Post-shutdown (the waiting period):

13. Monitor for 24 hours for unexpected errors in dependent services
14. Confirm no traffic attempts in logs
15. Update service catalog to show decommissioned status
16. Update architecture diagrams

Cleanup (after the grace period):

17. Delete Kubernetes resources
18. Archive or delete database (per retention policy)
19. Remove CI/CD pipelines
20. Archive source code repository (don't delete-you might need to reference it)
21. Remove monitoring dashboards
22. Cancel related cloud resources
23. Verify no ongoing charges with FinOps

The cleanup phase happens after a grace period — typically 30 days. This gives you time to discover any dependencies you missed and roll back if needed.

Automated Shutdown Script

The shutdown steps are repetitive and error-prone when done manually. Automate them, but always support a dry-run mode so you can verify what will happen before it happens.

shutdown-service.sh

```
#!/bin/bash
# Service shutdown script with dry-run support

set -euo pipefail

SERVICE_ID=$1
DRY_RUN=${2:-false}

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1"; }
run() {
  if [[ "$DRY_RUN" == "true" ]]; then
    log "[DRY RUN] Would execute: $*"
  else
    "$@"
  fi
}

log "Starting shutdown of $SERVICE_ID (dry_run=$DRY_RUN)"

# Save rollback state
log "Saving rollback state..."
REPLICAS=$(kubectl get deployment "$SERVICE_ID" -o jsonpath='{.spec.replicas}')
kubectl get ingress "$SERVICE_ID" -o yaml > "/tmp/${SERVICE_ID}-ingress-backup.yaml"

# Remove from load balancer
log "Removing from load balancer..."
run kubectl patch ingress "$SERVICE_ID" -p '{"spec":{"rules":[]}}'

# Scale to zero
log "Scaling deployment to zero..."
run kubectl scale deployment "$SERVICE_ID" --replicas=0

# Suspend cron jobs
log "Suspending cron jobs..."
for cron in $(kubectl get cronjob -l "app=$SERVICE_ID" -o name); do
  run kubectl patch "$cron" -p '{"spec":{"suspend":true}}'
done

# Revoke service account
log "Disabling service account..."
run kubectl delete serviceaccount "${SERVICE_ID}-sa" --ignore-not-found

```

```
43    log "Shutdown complete. Rollback state saved to /tmp/${SERVICE_ID}-*"
```

Bash shutdown script with dry-run mode and rollback state preservation.

Always run the script with `DRY_RUN=true` first, review the output, then run it for real. The few minutes this adds is worth it compared to the hours you'd spend fixing a botched shutdown.

Rollback Procedures

Sometimes the scream test screams loudly enough that you need to bring the service back. Having a tested rollback procedure is the difference between a 10-minute recovery and a 2-hour scramble.

The rollback reverses the shutdown steps in order: restore load balancer config, scale the deployment back up, unsuspend cron jobs, restore service account. Then wait for pods to be ready and verify health checks pass. Don't call it done until you've confirmed error rates in dependent services have returned to baseline and latency metrics look normal — health checks alone won't catch all problems.

> **⚠ WARNING**
>
> After a rollback, create an incident ticket even if there was no customer impact. You need to investigate why the decommissioning failed — was there an unknown consumer? Did someone ignore the deprecation warnings? The rollback buys you time, but you still need to solve the underlying problem.

Automatic rollback triggers should fire on clear signals: error rate in dependent services exceeding a threshold, critical alerts firing, or explicit rollback requests from on-call. Don't auto-rollback on every minor anomaly — you'll never finish decommissioning anything.

Data Handling During Decommissioning

Turning off the compute is easy. The data is where decommissioning gets complicated. You can't just delete everything — compliance requirements, potential rollback needs, and the possibility that someone will ask "can you get me that report from 2019?" all require careful planning.

Data Retention Strategy

Different data types have different retention requirements. PII (Personally Identifiable Information) and financial records often have legal retention periods (7 years for SOX (Sarbanes-Oxley Act) compliance, for example). Application logs might only need 90 days. Source code should probably be kept forever — storage is cheap, and you never know when you'll need to understand how something worked.

Data Type	Typical Retention	Archive Location	Notes
User PII	7 years	Encrypted cold storage	GDPR/CCPA require deletion on request
Transaction records	7 years	Encrypted cold storage	SOX, PCI-DSS requirements
Application logs	90 days	Log archive	Rarely needed after this
Configuration	1 year	Config archive	Useful for debugging similar systems
Source code	Forever	Archived repository	Storage is cheap, regret is expensive

Standard retention periods by data type.

For source code archival, use your Git host's archive feature (GitHub and GitLab both support marking repositories as archived, which makes them read-only) rather than deleting. If you must remove a repository entirely, clone it with full history to a tarball and store it in your artifact archive. You'll be glad you kept it when someone asks "how did the old system handle currency conversion?" two years later.

Before archiving, classify the data. If you're not sure what compliance requirements apply to a dataset, ask your legal or compliance team — this isn't something to guess at. Getting data retention wrong can mean regulatory fines or failing to produce records during litigation.

Database Archival Process

Database archival has a specific workflow that's easy to get wrong. The goal is to ensure data is accessible from the archive **before** you delete the source.

1. **Take a final backup**
   Include a clear label indicating this is the decommission backup. Use your standard backup tooling so restores work the same way.

2. **Verify the backup is restorable.**
   Actually restore it to a test environment and run validation queries. "We have a backup" means nothing if you can't restore from it.

3. **Export to a portable format.**
   Database backups are great for full restores, but if someone just needs to query old data, a Parquet or CSV export to S3 is more accessible. Include the schema definition.

4. **Create an archive manifest**
   Document what was archived, where it went, row counts by table, and instructions for accessing the data. Put this manifest somewhere discoverable-a wiki page or the service catalog entry for the decommissioned service.

5. **Schedule deletion with a grace period.**
   Don't delete the source database immediately. A 30-day grace period between archival and deletion has saved many teams from disaster.

archive-database.sh

```
#!/bin/bash
# Database archival with verification

set -euo pipefail

SERVICE_ID=$1
DB_NAME="${SERVICE_ID}_db"
ARCHIVE_BUCKET="s3://data-archive/${SERVICE_ID}"

# Take final backup
echo "Creating final backup..."
```

```
pg_dump "$DB_NAME" | gzip > "/tmp/${DB_NAME}_final.sql.gz"

# Export to Parquet for easier querying
echo "Exporting to Parquet..."
psql "$DB_NAME" -c "\copy (SELECT * FROM users) TO '/tmp/users.csv' CSV HEADER"
# ... repeat for each table

# Upload to archive
echo "Uploading to archive..."
aws s3 cp "/tmp/${DB_NAME}_final.sql.gz" "${ARCHIVE_BUCKET}/backup/"
aws s3 cp /tmp/*.csv "${ARCHIVE_BUCKET}/exports/"

# Verify archive is accessible
echo "Verifying archive..."
aws s3 ls "${ARCHIVE_BUCKET}/backup/${DB_NAME}_final.sql.gz" || exit 1

echo "Archive complete. Schedule database deletion for 30 days from now."
```

Database archival script with backup, export, and verification steps.

> **DANGER**
>
> Verification means more than checking files exist. Download a sample, decompress it, and run a query against the restored data. The archive that "succeeded" but wrote zero bytes has bitten more teams than I can count.

Post-Decommissioning

The service is off. The data is archived. But you're not done. Post-decommissioning work ensures nothing was missed, validates the cost savings, and captures lessons for the next time.

Cleanup Verification

Services leave behind more artifacts than you expect. A week after shutdown, run a verification sweep to confirm everything is actually gone.

#	Resource Type	Expected State	How to Verify
1	Kubernetes deployment	Deleted	`kubectl get deployment $SERVICE` returns NotFound
2	Kubernetes service	Deleted	`kubectl get service $SERVICE` returns NotFound
3	Database	Archived then deleted	Archive exists in S3, RDS instance terminated
4	DNS records	Deleted or redirected	`dig $SERVICE.example.com` returns NXDOMAIN
5	SSL certificates	Deleted	Certificate manager shows no active certs
6	Cloud resources	Deleted	Tag search returns zero results
7	CI/CD pipelines	Archived	Pipeline status shows archived
8	Monitoring	Deleted	No dashboards or alerts reference the service

Post-decommissioning cleanup verification checklist.

The most commonly missed items: DNS (Domain Name System) records (especially CNAMEs that point to load balancers), SSL certificates (which keep renewing automatically), and cloud resources tagged inconsistently (so they don't show up in tag-based searches). Check these explicitly.

If your cloud provider supports it, set up a budget alert for the service's cost allocation tag. If charges appear after decommissioning, something's still running.

Cost Savings Validation

Decommissioning takes effort. You need to prove the ROI to justify the next one.

Pull the service's cost data from the three months before decommissioning. Compare to the current cost (which should be just archive storage, close to zero). The difference is your monthly savings.

#	Cost Category	Before (Monthly)	After (Monthly)	Savings
1	Compute	$1,200	$0	$1,200
2	Storage	$300	$15 (archive)	$285
3	Network	$150	$0	$150
4	Licenses	$500	$0	$500
5	Total	$2,150	$15	$2,135

Example cost savings calculation for a decommissioned service.

Don't forget the hidden savings that are harder to quantify: one fewer service to patch during security updates, reduced audit scope for compliance reviews, no more false alerts waking up on-call, and less confusion for engineers trying to understand the architecture. These indirect benefits often exceed the direct cost savings.

Report the savings to leadership. This builds organizational support for future decommissioning efforts. "We saved $25,000/year by turning off legacy-user-service" is more compelling than "we should clean up old services."

Lessons Learned Documentation

Every decommissioning teaches you something. Capture it while it's fresh.

A retrospective document should cover:

- **Timeline comparison:**
  How long did you plan for vs. how long did it actually take? What caused delays?
- **Consumer discovery:**
  How many consumers did you know about beforehand? How many did you discover during the scream test? What discovery methods worked best?

- **Escalations:**
  How many escalations did you handle? What categories (extension requests, migration help, rollback)? Were any surprising?
- **What worked:**
  What would you do again? Early announcements? Automated rollback triggers? Weekly status updates?
- **What didn't work:**
  What would you do differently? Did you miss a communication channel? Was the scream test too aggressive? Too slow?
- **Surprises:**
  What caught you off guard? Undocumented integrations? Foreign key constraints? Batch jobs nobody knew about?

This documentation becomes the playbook for the next decommissioning. The engineer who handles it in six months will thank you.

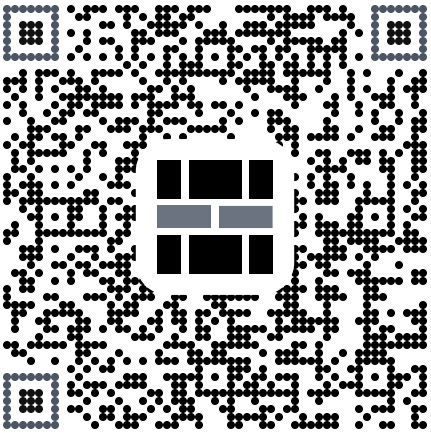
> **INFO**
>
> Calculate the ROI: divide annual cost savings by engineering hours spent times hourly rate. A typical decommissioning that saves $2,000/month and takes 40 engineering hours has a first-year ROI of 4x. That's a strong argument for prioritizing cleanup work.

Conclusion

The scream test — announce, degrade, fail, shutdown — is the safest way to discover unknown consumers before they become 3 AM incidents. Combine it with traffic analysis and distributed tracing to find dependencies proactively, not reactively.

Never delete data without verified archives and a 30-day grace period. And document every decommissioning so the next one goes faster. The organization that gets good at turning things off is the organization that can move quickly when building new things.

https://www.webstackbuilders.com/articles/service-decommissioning-scream-test-shutdown



Webstack Builders
1032 E. Brandon Boulevard, Suite 5230
Brandon, FL 33511

support@webstackbuilders.com
Toll Free (888) 987-1881
Local (302) 608-6864